

Lift Line Looks Database Design

Joshua Schwenk

Colorado State University Global

MIS407: Database Concepts

Dr. Shawn Harris

7/27/2025

Lift Line Looks Database Design

Located in Vail Village, Lift Line Looks (LLL) is a retailer specializing in retro ski and snowboard apparel. For years, the company has catered to tourists and locals from its flagship storefront. However, due to popular requests, LLL is embracing digital transformation by launching an e-commerce platform. Its design is to offer a seamless ordering experience for customers unable to purchase products at their physical location. To support this initiative, the company is building a relational database tailored to the needs of online retail.

The new database will serve as the backbone of the company's online operations, managing critical aspects such as customer information, employee assignments, product inventory, order processing, shipping logistics, and payments. Tables such as Customer, Orders, OrderItems, Product, Payment, and Shipping work together to facilitate accurate order tracking and reporting. By centralizing these operations in a structured PostgreSQL database, LLL can streamline order fulfillment while also generating meaningful analytics. These insights will help the company understand purchasing behavior, monitor inventory trends, improve customer service, and ultimately support its goal of expanding market reach without compromising the personalized experience it's known for.

Database Structure and Relationships

To support LLL's new online ordering system, a series of relational tables were developed using PostgreSQL. Figure 1 and Figure 2 document the SQL code used to create the foundational entities required for processing online orders, including Customer, Employees, Orders, Products, Payments, and Shipping. Each of these tables is designed with well-defined primary keys, appropriate data types, and business rules such as default values and constraints to

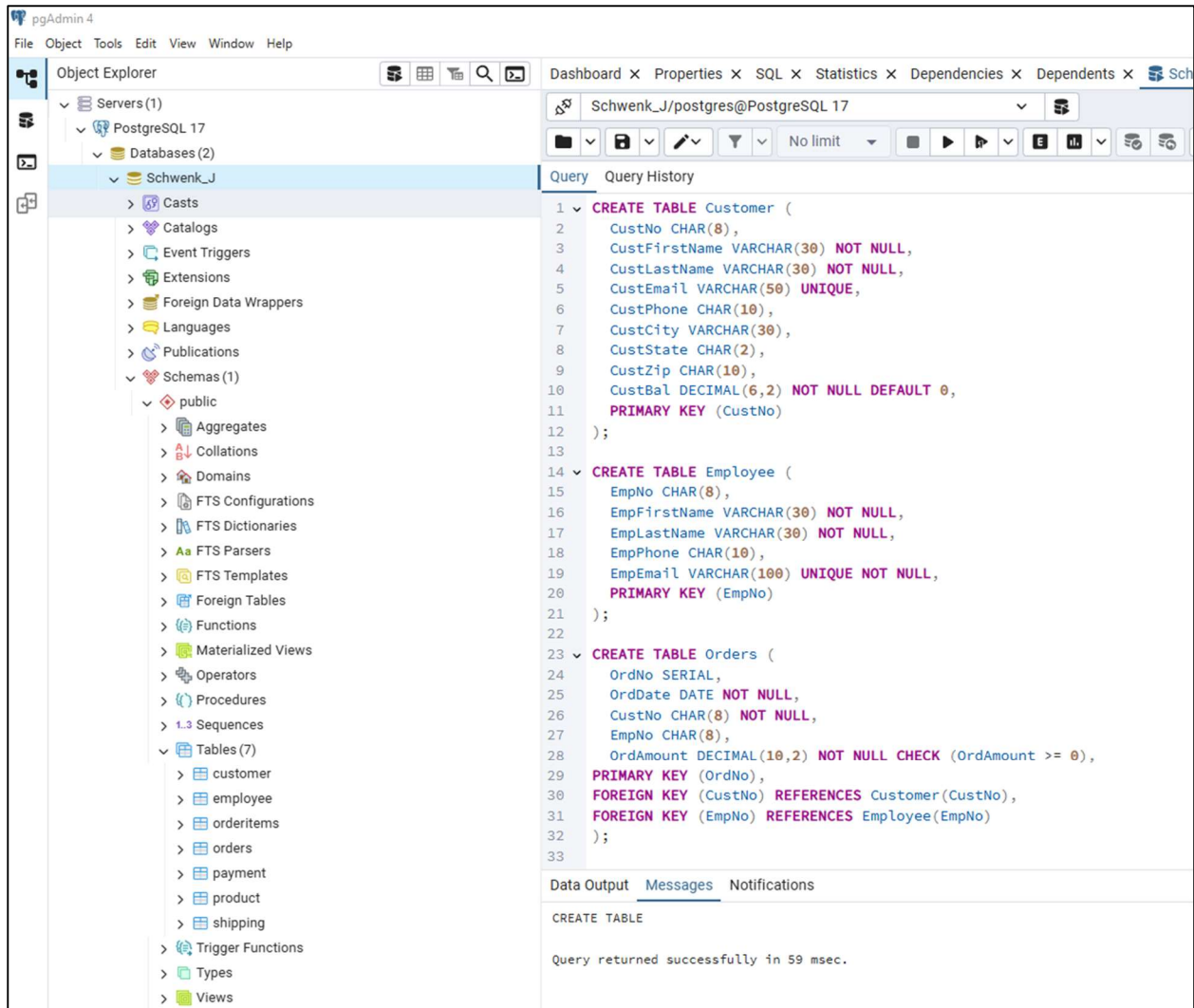
maintain data integrity. They also include foreign key relationships that enforce referential connections. For example, linking each order to a specific customer or employee.

In addition to these core entities, a separate table named OrderItems serves as a bridge between the Order and Product tables. As described by Mannino (2022), this design supports a many-to-many (M-N) relationship, allowing a single order to include multiple products and each product to appear in multiple orders. By normalizing this relationship through the OrderItems table, the database efficiently tracks item quantities, discounts, and product data without redundancy. This separation ensures that the system can scale with transaction volume while supporting accurate and flexible reporting.

While the original schema included a required EmpNo foreign key in the Orders Table, this assumption does not align with the operations of an online ordering system. In an e-commerce environment, customers place orders independently through the digital platform. To accommodate this difference, the database was adjusted to make the EmpNo field optional rather than mandatory. This schema allows orders to exist without a direct employee association while still supporting flexibility for scenarios for phone and in-store orders. This modification ensures the database remains realistic and scalable, accurately reflecting the business logic of LLL's online sales process.

Figure 1

SQL Table Creation for Customer, Employee, and Orders Tables



Note. This figure shows the SQL code used to create the Customer, Employee, and Orders tables for LLL's retail operations. The original code was modified to support the online ordering system.

Figure 2

SQL Table Creation for Product, Payment, Shipping, and OrderItems Entities

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure for 'Schwenk_J' on 'PostgreSQL 17'. The 'Tables (7)' folder is expanded, showing tables like 'customer', 'employee', 'orderitems', 'orders', 'payment', 'product', and 'shipping'. The main pane shows the SQL editor with the following code:

```

34 CREATE TABLE Product (
35     ProdNo VARCHAR(10),
36     ProdName VARCHAR(100),
37     ProdCategory VARCHAR(30),
38     ProdPrice DECIMAL(10,2),
39     ProdQuantity INTEGER,
40     PRIMARY KEY (ProdNo)
41 );
42
43 CREATE TABLE OrderItems (
44     OrdItemNo SERIAL,
45     OrdNo INTEGER NOT NULL,
46     ProdNo VARCHAR(10) NOT NULL,
47     OrdQuantity INTEGER NOT NULL CHECK (OrdQuantity > 0),
48     Discount DECIMAL(5,2) DEFAULT 0 CHECK (Discount >= 0),
49     PRIMARY KEY (OrdItemNo),
50     FOREIGN KEY (OrdNo) REFERENCES Orders(OrdNo),
51     FOREIGN KEY (ProdNo) REFERENCES Product(ProdNo)
52 );
53
54 CREATE TABLE Payment (
55     PayNo SERIAL,
56     OrdNo INTEGER NOT NULL,
57     PayDate DATE NOT NULL,
58     PayMethod VARCHAR(30) NOT NULL,
59     PayStatus VARCHAR(20) NOT NULL DEFAULT 'Pending',
60     PayAmount DECIMAL (10,2) NOT NULL CHECK (PayAmount >= 0),
61     PRIMARY KEY (PayNo),
62     FOREIGN KEY (OrdNo) REFERENCES Orders(OrdNo)
63 );
64
65 CREATE TABLE Shipping (
66     ShipNo SERIAL,
67     OrdNo INTEGER NOT NULL,
68     ShipDate DATE,
69     ShipMethod VARCHAR(30) NOT NULL,
70     ShipAddress VARCHAR(100) NOT NULL,
71     ShipCity VARCHAR(30) NOT NULL,
72     ShipState CHAR(2) NOT NULL,
73     ShipZip CHAR(10) NOT NULL,
74     ShipStatus VARCHAR(20) NOT NULL DEFAULT 'Pending',
75     TrackingNo VARCHAR(30),
76     PRIMARY KEY (ShipNo),
77     FOREIGN KEY (OrdNo) REFERENCES Orders(OrdNo)
78 );

```

The bottom pane shows the 'Data Output' tab with the message: 'CREATE TABLE' and 'Query returned successfully in 59 msec.'

Note. This figure displays the SQL code used to create the Product, Payment, Shipping, and OrderItems tables, which expands LLL’s database to support online ordering. These entities handle inventory management, transactional processing, fulfillment, and detailed order information. The OrderItems table functions as a bridge entity to resolve the M-N relationship between Orders and Product.

Using Database for Business Insights

One of the key advantages of implementing a relational database for LLL is the ability to transform data into actionable insights. By analyzing the relationship between customers, orders, products, payments, and shipping, the company can uncover patterns that inform strategic decisions. Below are six example business questions that demonstrate how the database can support reporting and analytics:

1. Which products are the top sellers during peak ski season (e.g., December to February)?

Using data from the OrderItems and Orders tables filtered by order dates between December and February, LLL can group product sales by ProdNo and count total quantities sold. This helps identify which retro items are most popular during the high season.

2. What is the average order value by customer ZIP code or region?

By joining the Orders and Customer tables, the database can group order totals (OrdAmount) by ZIP code or state to calculate the average spend per customer location. This insight helps drive targeted marketing.

3. Which customers make repeat purchases within a given year?

Using the Orders table, grouped by CustNo, LLL can count the number of orders per customer and filter for those with more than one order in the same year. This identifies loyal customers and supports retention strategies.

4. What is the distribution of shipping methods selected by customers?

The Shipping table contains the ShipMethod field. A simple group-by query can count the number of times each method (e.g., Standard, Express) is selected. This helps LLL evaluate shipping preferences and negotiate better carrier rates.

5. Which product categories generate the highest revenue?

By joining OrderItems with Product and calculating total revenue per ProdCategory (price × quantity – discount), LLL can determine which product lines contribute most to overall sales.

6. What percentage of orders are paid with each payment method?

The Payment table includes the PayMethod field. Grouping by payment method and dividing each count by the total number of payments will reveal the percentage breakdown. This helps LLL optimize the checkout experience and consider expanding or streamlining payment options.

Conclusion

Lift Line Looks (LLL) is taking an important step by expanding into online ordering to meet the changing needs of its customers. The PostgreSQL database designed for this purpose supports the company's shift by organizing essential information such as customer details, product inventory, orders, payments, and shipping. Together, these tables form a functional system that allows for accurate order tracking and efficient processing.

In addition to helping with day-to-day operations, the database also makes it possible to generate useful business insights. By examining patterns in the data, LLL can learn more about customer behavior, sales trends, and shipping preferences. This information can support future decisions and help the company improve its service. As online shopping continues to grow, having a well-designed database puts LLL in a better position to serve both new and existing customers without losing sight of what makes the brand unique.

References

Mannino, M. (2022). *Database design: Design, query, formulation, and administration using Oracle and PostgreSQL*. SAGE Publications.